UNIVERSITY OF MYSORE

A Mini Project Report on

"Python-Based Digital Image Processing Application With 3D Visualization"

Submitted in partial fulfilment for the award of degree of Bachelor of Computer Applications during the year 2025-2026.

SUBMITTED BY

NISARGA NS (U01BP24S0087)

Under the Guidance

Prof. Nethra G

Department of BCA

DEPARTMENT OF COMPUTER APPLICATIONS

GSSS SIMHA SUBBAMAHALAKSHMI FIRST GRADE COLLEGE

KRS Road, Metagalli, Mysuru-570016

ACKNOWLEDGEMENT

I sincerely owe my gratitude to all the persons who helped and guided me in completing this project.

I sincerely thank **Mrs. Anupama B Pandit,** Secretary**,** GSSS SSFGC**,** Mysore, for her continuous enthusiasm and support.

I am thankful to **Dr. Rakesh T S**, Principal GSSS SSFGC, Mysore for having supported me in my academic endeavours.

I would like to sincerely thank my project guide **Prof. Nethra G**, Department of Computer Applications for providing relevant information, valuable guidance and encouragement to complete this project.

I would also like to thank all our teaching and non-teaching staff members of the department.

Lastly, I would like to thank my parents and friends for their support, encouragement and guidance throughout the project.

GSSS SIMHA SUBBAMAHALAKSHMI FIRST GRADE COLLEGE

KRS Road, Metagalli, Mysore-570016

DEPARTMENT OF COMPUTER APPLICATIONS

CERTIFICATE

Certified that the mini project work entitled "*Python-Based Digital Image Processing Application with 3D Visualization*" is a project work carried out by *NISARGA NS (U01BP24S0087)* in partial fulfilment for the award of degree of Bachelor of Computer Applications and University of Mysore, Mysuru during the year 2025-2026. The mini project report has been approved as it satisfies the academic requirements with respect to the Project Work prescribed for Bachelor of Computer Applications degree.

Signature of Guide
Prof. Nethra G
Assistant Professor

Signature of Principal
Dr. Rakesh TS

Signature of HOD
Mrs. Nethra G

ABSTRACT

This project presents a **Python-Based Digital Image Processing Application with 3D Visualization**, designed to provide an interactive and user-friendly platform for performing various image processing operations. The application is developed using Python and integrates libraries such as **OpenCV, NumPy, Tkinter, Pillow, and Matplotlib** to deliver efficient image manipulation and visualization functionalities.

The system allows users to upload images and perform multiple operations including grayscale conversion, edge detection, and application of various filters such as Snapchat-style smoothing, Instagram warm effect, black and white enhancement, and cool tone adjustment. These features enhance the visual quality of images and simulate real-world social media editing tools.

A key feature of the application is the generation of a 3D surface visualization from a 2D image. This is achieved by converting image intensity values into a height map, which is then rendered as a rotating 3D surface, providing a deeper understanding of image structure and texture.

Additionally, the application includes a creative sticker generation module that allows users to produce customized images with decorative frames and text. The graphical user interface, developed using Tkinter, ensures ease of use and improves user interaction through organized controls and real-time feedback.

Overall, this project demonstrates the effective use of digital image processing techniques combined with visualization and GUI design to create a practical and visually appealing application. It can be further extended with advanced features such as real-time processing, additional filters, and integration of intelligent image analysis techniques.

TABLE OF CONTENT

CHAPTER 1

INTRODUCTION

Digital image processing plays an important role in various fields such as photography, medical imaging, computer vision, and multimedia applications. It involves the manipulation and analysis of images using computational techniques to enhance their quality and extract useful information. With the increasing use of digital devices and social media platforms, the demand for efficient and user-friendly image processing tools has grown significantly.

This project, titled **"Python-Based Digital Image Processing Application with 3D Visualization"**, focuses on developing an interactive application that allows users to perform essential image processing operations in a simple and effective manner. The system is implemented using Python and integrates various libraries to handle image manipulation, graphical user interface design, and data visualization.

The application enables users to upload images and apply different processing techniques such as grayscale conversion, edge detection, and visual filters that enhance the appearance of images. In addition to basic processing, the project introduces a 3D visualization feature, where a 2D image is transformed into a three-dimensional surface representation. This helps in understanding the intensity and structural details of an image in a more intuitive way.

Furthermore, the application includes a creative module for generating customized stickers with decorative frames and text, making it both functional and visually appealing. The graphical user interface ensures ease of use and provides real-time interaction with the system.

Overall, this project demonstrates how digital image processing techniques can be combined with visualization and user interface design to create a practical and engaging software application.

PROBLEM STATEMENT

Many existing image editing tools are complex, require technical knowledge, or do not provide all features in a single platform. Users often need multiple applications to perform basic image processing, apply filters, and create visual outputs like 3D representations or customized designs.

Therefore, there is a need for a simple and user-friendly application that integrates image processing, visualization, and creative features in one system. This project aims to develop a Python-based application that allows users to easily process images, apply filters, generate 3D visualizations, and create customized stickers through an interactive graphical interface.

OBLECTIVES

- To develop a user-friendly application for digital image processing using Python.
- To enable users to upload and display images through a graphical user interface.
- To implement basic image processing techniques such as grayscale conversion and edge detection.
- To provide various image filters for enhancing visual appearance.
- To generate 3D visualization from 2D images for better understanding of image structure.
- To create a feature for generating customized stickers with frames and text.
- To integrate all functionalities into a single, interactive system.

CHAPTER 2

Existing System and Proposed System

Existing System

In the current scenario, image processing is mainly performed using advanced software tools such as Photoshop and other editing applications. While these tools provide a wide range of features, they are often complex, require technical knowledge, and may not be user-friendly for beginners. Additionally, many tools focus only on editing and lack integrated features like 3D visualization or creative outputs such as customized stickers.

Users often need to switch between multiple applications to perform basic image processing tasks, apply filters, and generate different types of visual outputs. This increases complexity and reduces efficiency, especially for users who require simple and quick solutions.

Therefore, the existing systems do not provide a simple, unified, and interactive platform that combines image processing, visualization, and creative features in a single application.

Limitations

- The application works only on a local system and does not support online or cloud-based processing.
- It processes only static images and does not support video or real-time camera input.
- The image size is fixed (500×500), which may affect image quality and flexibility.
- The available filters are limited and may not match advanced editing tools.
- The 3D visualization is based on basic height mapping and may not provide highly accurate depth representation.
- The application does not include advanced techniques such as artificial intelligence or machine learning.
- Performance may decrease when handling large or high-resolution images.

Proposed Solution

The proposed system is a **Python-Based Digital Image Processing Application with 3D Visualization**, designed to provide a simple and integrated platform for performing various image processing operations. The system overcomes the limitations of existing tools by offering multiple functionalities within a single user-friendly interface.

The application allows users to upload images and apply essential processing techniques such as grayscale conversion, edge detection, and various visual filters. It also includes a 3D visualization feature that converts a 2D image into a height map and displays it as a rotating 3D surface for better understanding of image structure.

In addition, the system provides a creative feature for generating customized stickers with frames and text. The graphical user interface ensures easy interaction, enabling users to perform all operations efficiently without requiring advanced technical knowledge.

Overall, the proposed system offers a simple, interactive, and efficient solution by combining image processing, visualization, and creative design features in a single application.

CHAPTER 3

Project Design

The **system architecture** of the project is designed to provide a smooth flow of data from user input to processed output through an interactive graphical interface. The application follows a modular structure where each component performs a specific function.

The process begins with the user uploading an image through the graphical user interface. The input image is then passed to the image processing module, where various operations such as grayscale conversion, edge detection, and filter application are performed using Python libraries.

The processed image is displayed in the output section of the interface in real time. For advanced functionality, the system includes a 3D visualization module, which converts the image into a height map and renders it as a rotating 3D surface. Additionally, a sticker generation module creates customized outputs with frames and text.

All modules are integrated within the GUI, ensuring easy navigation and interaction. The system architecture ensures efficient processing, clear data flow, and a user-friendly experience.

Overall, the design of the system focuses on simplicity, modularity, and effective integration of image processing and visualization features.

CHAPTER 4

Implementation and Results

Working Process

1. The user starts the application, and the graphical user interface (GUI) is displayed.
2. The user uploads an image using the file selection option provided in the interface.
3. The system reads the image and resizes it to a standard dimension for processing.
4. Preprocessing is performed on the image, which includes grayscale conversion and noise reduction using Gaussian blur.
5. Edge detection is applied to identify boundaries and important features in the image.
6. The user can select different operations such as viewing the original image, grayscale image, or edge-detected image.
7. Various filters like warm tone, cool tone, black-and-white, and smoothing effects can be applied to enhance the image.
8. For advanced functionality, the system converts the image into a height map and generates a rotating 3D visualization.
9. The user can create a customized sticker by applying a circular mask, adding a frame, and inserting text.
10. The final sticker is saved as an image file in the system.
11. The processed results are displayed in real time within the GUI, along with popup messages for user interaction.
12. The user can exit the application after completing the required operations.

Code:

```
import cv2
import numpy as np
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
import matplotlib.pyplot as plt

# Global variables
image = None
gray_3ch = None
edges_3ch = None
is_running = False

# Convert image
def convert_img(img):
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    return ImageTk.PhotoImage(Image.fromarray(img))

def show_image(img):
    img_tk = convert_img(img)
    panel.config(image=img_tk)
    panel.image = img_tk

# Popup
def show_popup(msg):
    popup.config(text=msg)
    popup.place(relx=0.5, rely=0.12, anchor="n")
    root.after(2000, lambda: popup.place_forget())

# Upload
def upload_image():
    global image, gray_3ch, edges_3ch

    path = filedialog.askopenfilename()
    if not path:
        return

    image = cv2.imread(path)
    image = cv2.resize(image, (500, 500))

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (5,5), 0)
```

```
    edges = cv2.Canny(gray, 100, 200)

    gray_3ch = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
    edges_3ch = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

    show_image(image)
    enable_buttons()
    show_popup("Image Uploaded ✓")

# Enable buttons
def enable_buttons():
    for b in action_buttons:
        b.config(state="normal")

def disable_buttons():
    for b in action_buttons:
        b.config(state="disabled")

# Basic views
def show_original():
    show_image(image)
    show_popup("Original")

def show_gray():
    show_image(gray_3ch)
    show_popup("Grayscale")

def show_edges():
    show_image(edges_3ch)
    show_popup("Edges")

# 📸 Filters
def snapchat_filter():
    img = cv2.bilateralFilter(image, 9, 75, 75)
    img = cv2.convertScaleAbs(img, alpha=1.2, beta=30)
    show_image(img)
    show_popup("Snapchat Filter 📸")

def insta_warm():
    img = image.astype(np.float32)
    img[:,:,2] += 40
    img[:,:,1] += 20
    img = np.clip(img, 0, 255).astype(np.uint8)
    show_image(img)
```

```
    show_popup("Instagram Warm 🌈")

def insta_bw():
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    bw = cv2.convertScaleAbs(gray, alpha=1.5, beta=20)
    show_image(cv2.cvtColor(bw, cv2.COLOR_GRAY2BGR))
    show_popup("B&W ❤️")

def cool_filter():
    img = image.astype(np.float32)
    img[:,:,0] += 40
    img = np.clip(img, 0, 255).astype(np.uint8)
    show_image(img)
    show_popup("Cool Tone 💙")

# 🤯 3D Model
def show_3d():
    global is_running
    if image is None or is_running:
        return

    is_running = True
    show_popup("Generating 3D...")

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 100, 200)
    height_map = (gray * 0.6 + edges * 0.4) / 255.0

    x = np.linspace(0, 1, height_map.shape[1])
    y = np.linspace(0, 1, height_map.shape[0])
    x, y = np.meshgrid(x, y)

    fig = plt.figure(figsize=(5,5))
    ax = fig.add_subplot(111, projection='3d')

    angle = 0

    def animate():
        nonlocal angle
        global is_running

        if not is_running:
            plt.close(fig)
            show_popup("Ready ✓")
```

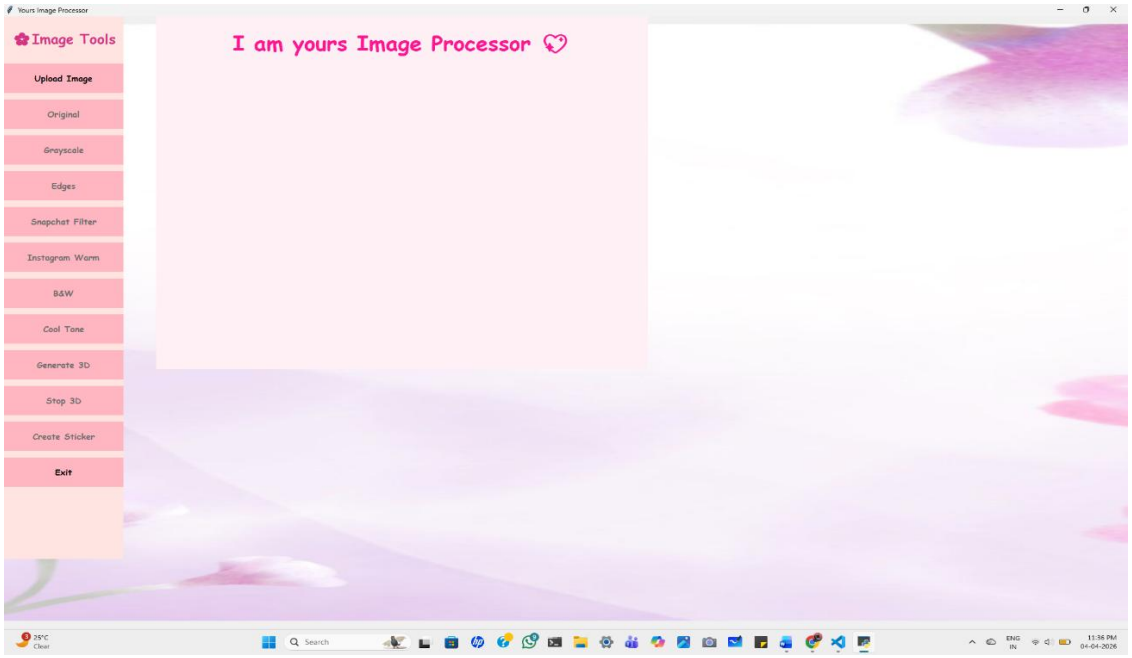
```
            return

        ax.clear()
        ax.plot_surface(x, y, height_map, cmap='plasma')
        ax.axis('off')
        ax.view_init(30, angle)

        fig.canvas.draw()
        img_3d = np.asarray(fig.canvas.buffer_rgba())
        img_3d = cv2.cvtColor(img_3d, cv2.COLOR_RGBA2BGR)

        show_image(img_3d)

        angle += 10
        root.after(100, animate)

    animate()

def stop_3d():
    global is_running
    is_running = False
    show_popup("3D Stopped")

# 🎁 Sticker (FINAL)
def create_sticker():
    if image is None:
        return

    mask = np.zeros((500, 500), dtype=np.uint8)
    cv2.circle(mask, (250, 250), 200, 255, -1)

    result = cv2.bitwise_and(image, image, mask=mask)

    bg = np.ones_like(image) * 255
    inv = cv2.bitwise_not(mask)
    bg = cv2.bitwise_and(bg, bg, mask=inv)

    final = cv2.add(result, bg)

    border_color = (180,105,255)
    framed = cv2.copyMakeBorder(final, 40, 80, 40, 40,
                                cv2.BORDER_CONSTANT,
                                value=border_color)
```

```
    text = "Thank You, Visit Again"
    font = cv2.FONT_HERSHEY_SIMPLEX
    scale = 1.2
    thickness = 3

    (tw, th), _ = cv2.getTextSize(text, font, scale, thickness)
    x = (framed.shape[1] - tw) // 2
    y = framed.shape[0] - 30

    # Background box
    cv2.rectangle(framed,
                  (x-10, y-th-10),
                  (x+tw+10, y+10),
                  (255,182,193),
                  -1)

    cv2.putText(framed, text, (x, y),
                font, scale, (147,20,255),
                thickness, cv2.LINE_AA)

    cv2.imwrite("sticker.png", framed)
    show_popup("Sticker Saved 🎁")

# Button style
def create_button(frame, text, command):
    btn = tk.Button(frame, text=text, command=command,
                    font=("Comic Sans MS", 11, "bold"),
                    bg="#ffb6c1", fg="black",
                    width=22, height=2, bd=0)
    btn.pack(pady=6)
    return btn

# GUI
root = tk.Tk()
root.title("Yours Image Processor")
root.geometry("1100x650")

# Background
bg = Image.open("rose_bg.jpg").resize((2600,1100))
bg_photo = ImageTk.PhotoImage(bg)
tk.Label(root, image=bg_photo).place(x=0,y=0,relwidth=1,relheight=1)

# Sidebar
sidebar = tk.Frame(root, bg="#ffe4e1", width=260)
```

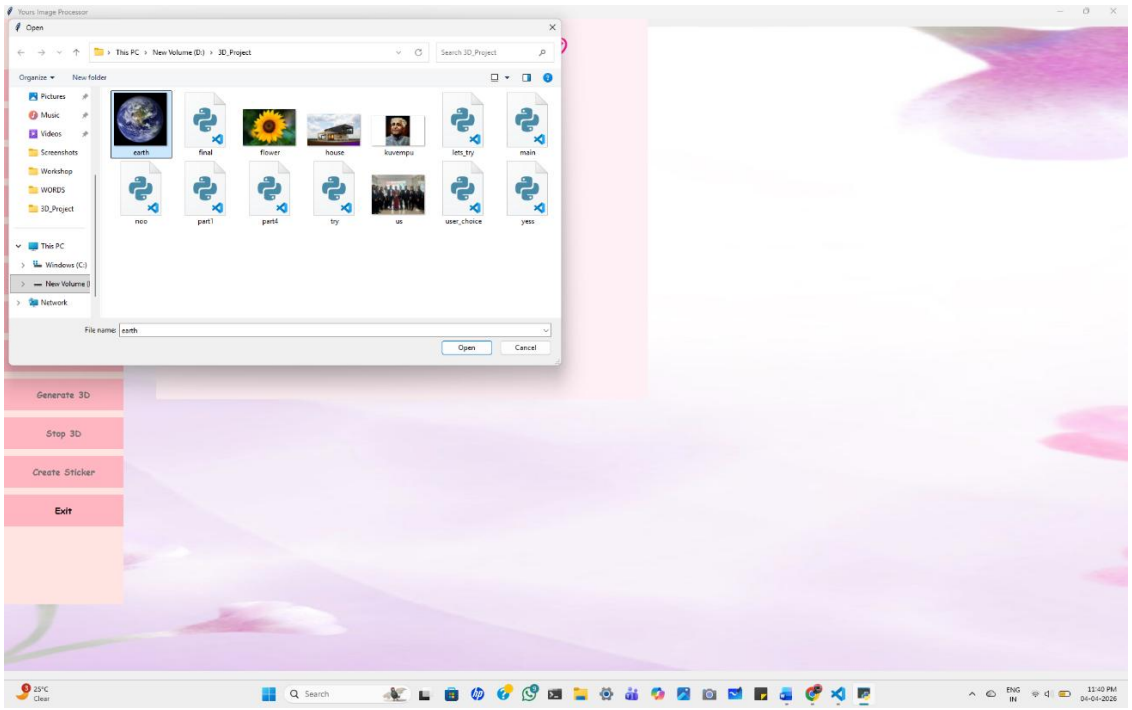
```
sidebar.place(x=0,y=0,height=1000)

tk.Label(sidebar, text="✿ Image Tools",
        font=("Comic Sans MS",18,"bold"),
        fg="#d63384", bg="#ffe4e1").pack(pady=20)

create_button(sidebar,"Upload Image",upload_image)
btn_original=create_button(sidebar,"Original",show_original)
btn_gray=create_button(sidebar,"Grayscale",show_gray)
btn_edges=create_button(sidebar,"Edges",show_edges)

# Filters
btn_snap=create_button(sidebar,"Snapchat Filter",snapchat_filter)
btn_warm=create_button(sidebar,"Instagram Warm",insta_warm)
btn_bw=create_button(sidebar,"B&W",insta_bw)
btn_cool=create_button(sidebar,"Cool Tone",cool_filter)

btn_3d=create_button(sidebar,"Generate 3D",show_3d)
btn_stop=create_button(sidebar,"Stop 3D",stop_3d)
btn_sticker=create_button(sidebar,"Create Sticker",create_sticker)
create_button(sidebar,"Exit",root.quit)

action_buttons=[btn_original,btn_gray,btn_edges,
                btn_snap,btn_warm,btn_bw,btn_cool,
                btn_3d,btn_stop,btn_sticker]
disable_buttons()

# Main area
main=tk.Frame(root,bg="#fff0f5")
main.place(x=260,y=0,width=840,height=650)

tk.Label(main,text="I am yours Image Processor 💖",
        font=("Comic Sans MS",28,"bold"),
        fg="#ff1493",bg="#fff0f5").pack(pady=20)

panel=tk.Label(main,bg="#fff0f5")
panel.pack(expand=True)

popup=tk.Label(main,font=("Comic Sans MS",12,"bold"),
               bg="#ff69b4",fg="white",padx=20,pady=5)

root.mainloop()
```

OUTPUT:

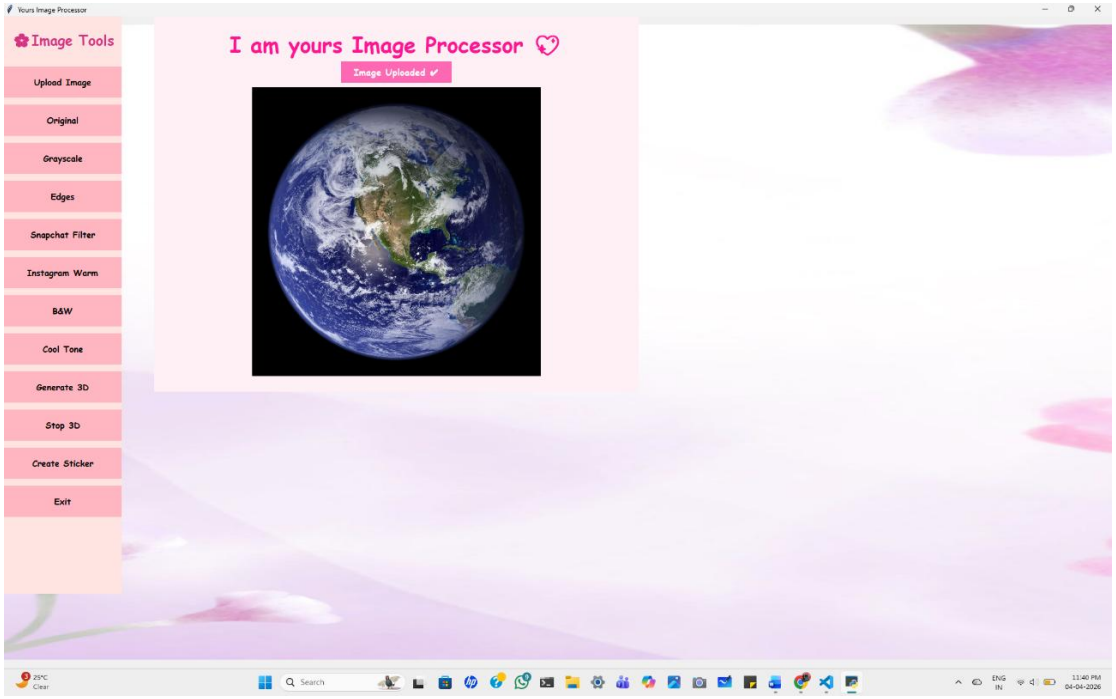
1. User Interface (Image Processor Website) [Fig.1.1]



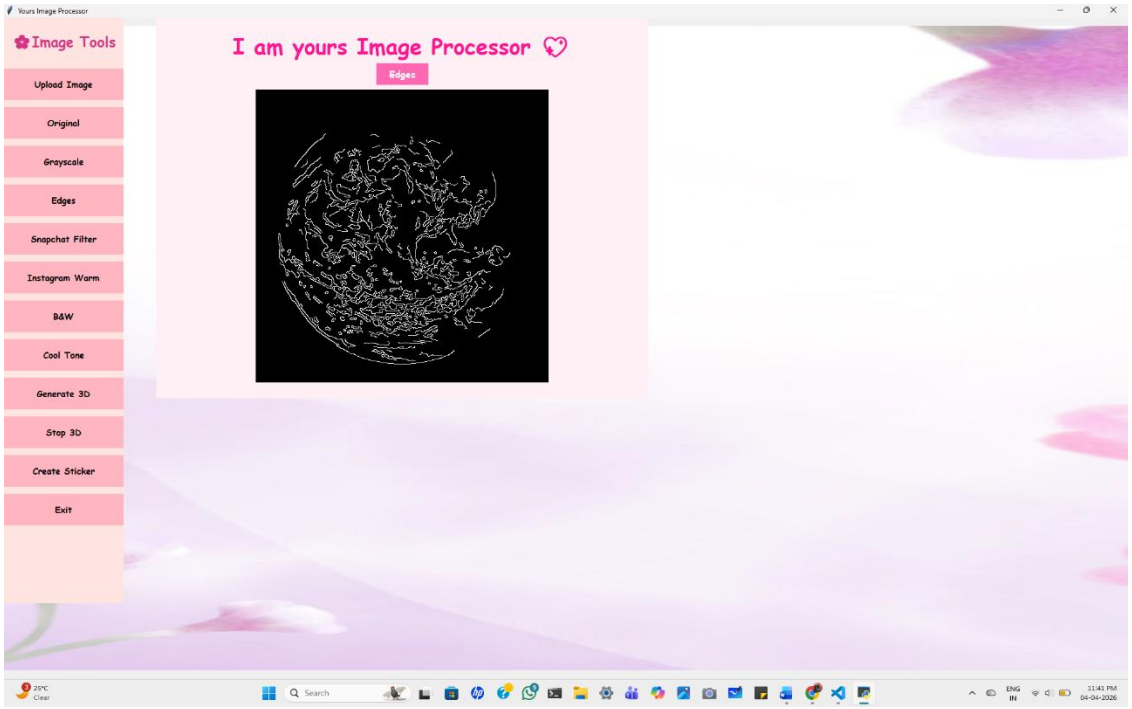
2. Uploading Image (From Explorer) [Fig.2.2]



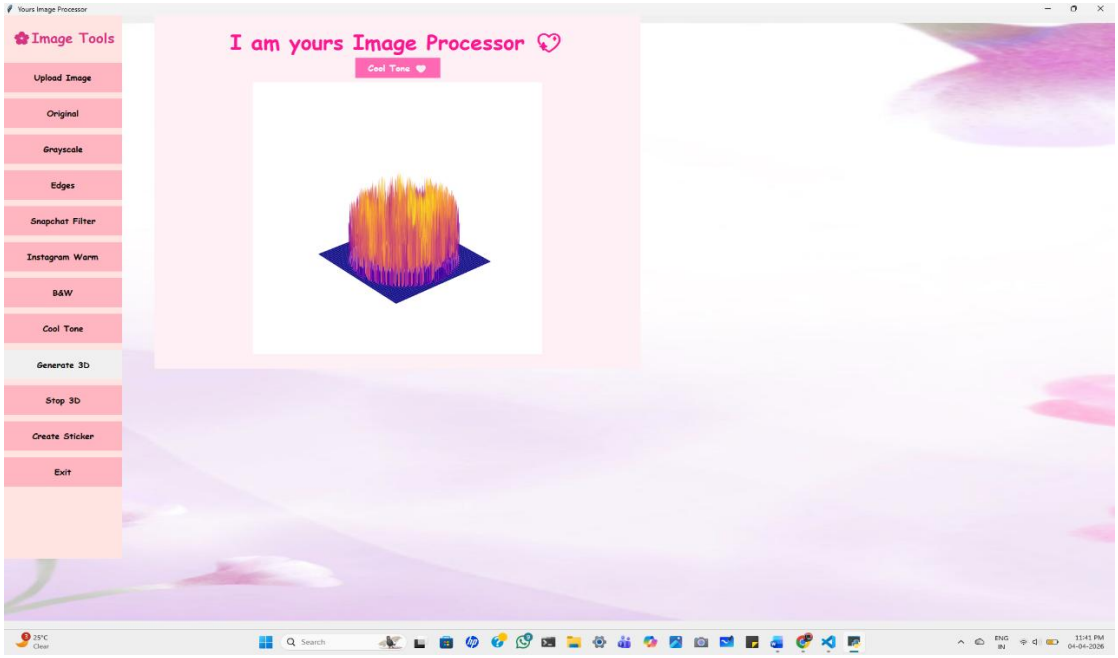
4. **Image Uploaded Message [Fig.2.3]**



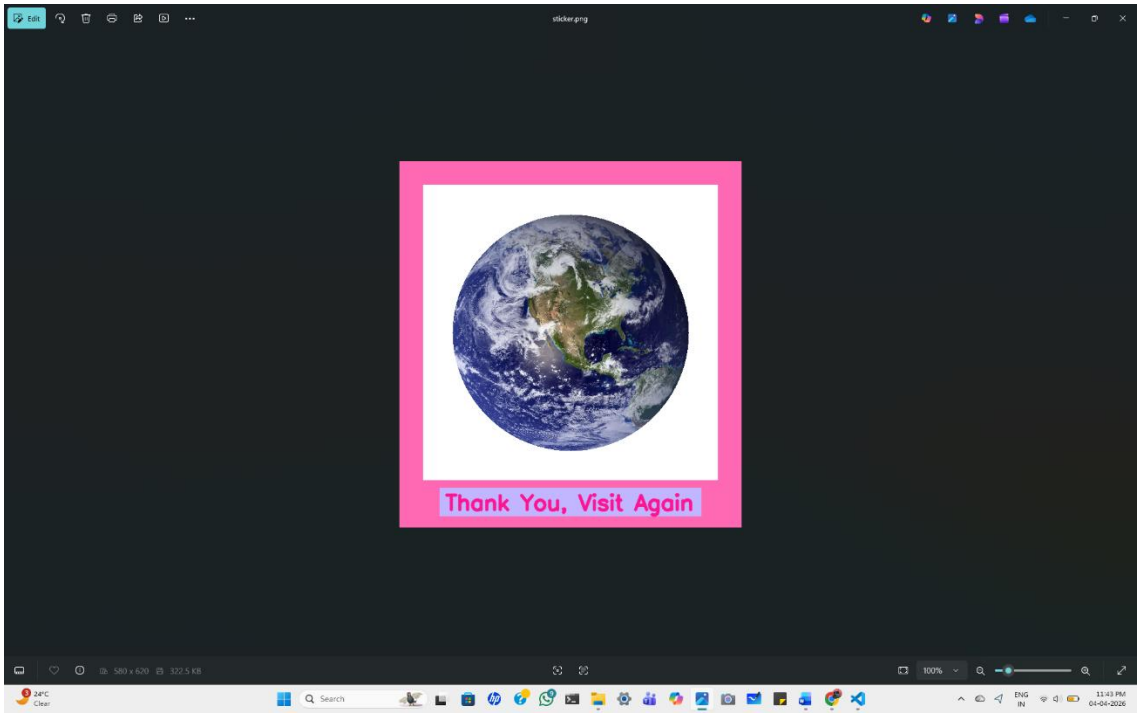
3. Processed Image (Original Image to Grey) [Fig.2.4]



5. 3D Visualization of the Image [Fig.2.5]



6. Sticker Generation (Downloaded to device) [Fig.2.6]



Results

The developed application was successfully implemented and tested for various image processing operations. The system was able to load and display images through the graphical user interface without errors. All basic image processing functions such as grayscale conversion and edge detection were executed accurately, producing clear and expected outputs.

The implemented filters, including warm tone, cool tone, black-and-white, and smoothing effects, enhanced the visual appearance of images effectively. The outputs were displayed in real time, ensuring a smooth user experience.

The 3D visualization feature successfully converted 2D images into height maps and rendered them as rotating 3D surfaces. This provided a better understanding of image structure and intensity variations.

The sticker generation module produced customized images with circular cropping, colored frames, and text. The final output was successfully saved as an image file.

Overall, the system performed efficiently, and all modules worked as expected, demonstrating the successful implementation of image processing, visualization, and GUI functionalities.

CHAPTER 5

Conclusion And Future Scope

Summary

This project, titled **"Python-Based Digital Image Processing Application with 3D Visualization"**, focuses on developing a user-friendly application for performing various image processing operations. The system allows users to upload images and apply techniques such as grayscale conversion, edge detection, and different visual filters to enhance image quality.

A key feature of the project is the 3D visualization of images, where a 2D image is converted into a height map and displayed as a rotating 3D surface. Additionally, the application includes a sticker generation feature that creates customized images with frames and text, making the system both functional and creative.

The graphical user interface ensures easy interaction and real-time display of results, improving usability for users with minimal technical knowledge. Overall, the project successfully demonstrates the integration of image processing, visualization, and GUI design into a single efficient application.

Project's Based Learning Outcome

From this project, I learned how to develop a complete image processing application using Python. We gained knowledge in using libraries like OpenCV, NumPy, Tkinter, Pillow, and Matplotlib for image manipulation and GUI development.

I understood how to implement image processing techniques such as grayscale conversion, edge detection, and applying filters. We also learned how to create 3D visualizations from 2D images and generate customized outputs like stickers.

Additionally, I improved my skills in designing user-friendly interfaces, integrating multiple functionalities, and handling real-time user interactions. Overall, the project enhanced our practical knowledge, programming skills, and understanding of digital image processing concepts.

Future Scope

The proposed system can be further enhanced by adding advanced features and improving its functionality. In the future, real-time image processing using a webcam can be implemented to make the application more interactive. Additional filters and editing tools can be included to provide a wider range of image enhancement options.

The system can also be extended to support high-resolution images and different file formats without resizing limitations. Integration of advanced techniques such as face detection, object recognition, and background removal can improve the intelligence and usability of the application.

Furthermore, the application can be converted into a web-based or mobile application to increase accessibility and usability across different platforms. Performance optimization and user interface improvements can also be made to enhance the overall user experience.

Overall, the project has significant potential for expansion by incorporating advanced technologies and improving its scalability and efficiency.